

Guide d'Installation Complet — Setup 100% Gratuit

LLM local (Ollama) + VS Code + Docker + RAG + Dataset + GitHub

Vue d'ensemble de ton setup final

Brique	Outil	Rôle
Éditeur	VS Code	Ta base de travail
Assistant de code	Cline (extension VS Code) + Ollama	Autocomplétion et chat avec un LLM qui tourne sur TA machine, gratuit et illimité
Modèle de code local	qwen2.5-coder (via Ollama)	Le "cerveau" qui t'aide à coder
Modèle d'embeddings local	nomic-embed-text (via Ollama)	Pour ton RAG (Agent 4), gratuit et illimité
Conteneurisation	Docker Desktop	Fait tourner ton infrastructure
Complément cloud gratuit	GitHub Copilot Free + Claude.ai (chat web)	Pour varier et pour comprendre des concepts

ÉTAPE 1 — Installer VS Code

1. Va sur <https://code.visualstudio.com/>
2. Télécharge et installe (réglages par défaut).

✅ Terminé quand : VS Code s'ouvre normalement.

ÉTAPE 2 — Installer Git et créer ton dépôt GitHub

2.1 — Installer Git

```
bash

# Vérifie si c'est déjà installé
git --version
```

Si erreur : télécharge sur <https://git-scm.com/downloads> et installe.

2.2 — Configurer ton identité

```
bash

git config --global user.name "Sahar"
git config --global user.email "ton-email@example.com"
```

2.3 — Créer le dépôt sur GitHub

1. Va sur <https://github.com>, connecte-toi (crée un compte si besoin).
2. Clique "New repository" → nomme-le (ex : `plateforme-iot-devops-ai`) → coche "Add a README" → "Create repository".

2.4 — Cloner le dépôt sur ta machine

Copie le lien du dépôt (bouton vert "Code" → HTTPS), puis dans un terminal :

```
bash

cd Documents # ou l'endroit où tu veux ranger ton projet
git clone https://github.com/ton-compte/plateforme-iot-devops-ai.git
cd plateforme-iot-devops-ai
```

2.5 — Ouvrir le dossier dans VS Code

```
bash

code .
```

✅ **Terminé quand :** VS Code s'ouvre directement sur ton dossier de projet, avec l'icône Git visible dans la barre de gauche.

ÉTAPE 3 — Installer Docker Desktop

1. Va sur <https://www.docker.com/products/docker-desktop/>
2. Télécharge la version pour ton système d'exploitation.
3. Installe (réglages par défaut). Sur Windows, ça peut te demander d'activer WSL2 — accepte, Docker Desktop guide l'installation automatiquement.
4. Lance Docker Desktop, attends que l'icône (en bas à droite ou dans la barre de menu) indique qu'il tourne (icône fixe, pas animée).
5. Vérifie dans le terminal :

```
bash
```

```
docker --version
docker run hello-world
```

✅ **Terminé quand :** un message de bienvenue s'affiche.

ÉTAPE 4 — Installer Ollama (ton LLM local)

4.1 — Télécharger et installer Ollama

- **Windows/Mac :** va sur <https://ollama.com/download>, télécharge l'installeur, lance-le.
- **Linux :**

```
bash

curl -fsSL https://ollama.com/install.sh | sh
```

4.2 — Vérifier l'installation

```
bash

ollama --version
```

4.3 — Choisir ton modèle selon la RAM de ta machine

📘 **RAM nécessaire :** vérifie la RAM de ton PC (Windows : clic droit sur "Ce PC" → Propriétés ; Mac : menu Pomme → À propos de ce Mac).

Ta RAM disponible	Modèle à installer	Commande
8 Go	qwen2.5-coder:7b (léger, bon pour débuter)	<code>ollama pull qwen2.5-coder:7b</code>
16 Go	qwen2.5-coder:14b (meilleure qualité)	<code>ollama pull qwen2.5-coder:14b</code>
Moins de 8 Go	qwen2.5-coder:1.5b (très léger)	<code>ollama pull qwen2.5-coder:1.5b</code>

Pour ton RAG (Agent 4), installe aussi le modèle d'embeddings :

```
bash

ollama pull nomic-embed-text
```

■ **Modèle d'embeddings** : un modèle spécialisé, différent d'un modèle de chat — sa seule tâche est de transformer du texte en vecteurs numériques (voir Étape 6). C'est petit et rapide (~270 Mo).

🕒 Le téléchargement du modèle de code peut prendre 10-30 minutes selon ta connexion (c'est plusieurs Go).

4.4 — Tester ton modèle local

```
bash

ollama run qwen2.5-coder:7b
```

Une fois le prompt `>>>` apparaît, teste :

```
Écris une fonction Python qui additionne deux nombres.
```

✅ **Terminé quand** : le modèle répond avec du code, directement sur ta machine, sans connexion Internet nécessaire après le téléchargement.

Quitte avec `/bye`.

ÉTAPE 5 — Connecter Ollama à VS Code (extension Cline)

5.1 — Installer l'extension Cline

1. Dans VS Code, clique sur l'icône Extensions (Ctrl+Shift+X).
2. Cherche "Cline".
3. Installe l'extension (par "Cline Bot").

■ **Pourquoi Cline et pas Continue.dev** : Continue.dev reste utilisable mais n'est plus activement mis à jour depuis son rachat par Cursor. Cline est aujourd'hui l'extension la plus activement maintenue pour connecter VS Code à un modèle local.

5.2 — Configurer Cline pour utiliser Ollama

1. Clique sur l'icône Cline dans la barre latérale de VS Code.
2. Dans les réglages (icône engrenage), choisis comme provider : **Ollama**.
3. URL de base : `http://localhost:11434` (c'est l'adresse par défaut d'Ollama sur ta machine).
4. Modèle : sélectionne `qwen2.5-coder:7b` (ou la version que tu as installée).

✅ **Terminé quand** : tu peux écrire une demande dans le panneau Cline (ex : "crée un fichier test.py qui affiche bonjour") et voir le modèle local répondre et proposer une modification de fichier.

1. Cherche "GitHub Copilot" dans les extensions VS Code, installe-le.
 2. Connecte-toi avec ton compte GitHub — le palier gratuit (2000 complétions + 50 messages chat/mois) s'active automatiquement.
-

ÉTAPE 6 — Construire ton RAG avec des embeddings locaux (gratuit, illimité)

6.1 — Installer les bibliothèques Python

```
bash

pip install chromadb langchain-text-splitters ollama
```

6.2 — Générer des embeddings avec Ollama (au lieu d'une API payante)

```
python
```

```

import ollama
import chromadb

client_chroma = chromadb.PersistentClient(path="./rag_db")
collection = client_chroma.get_or_create_collection("docs_techniques")

def generer_embedding(texte: str) -> list:
    reponse = ollama.embeddings(model="nomic-embed-text", prompt=texte)
    return reponse["embedding"]

def indexer_document(chemin_fichier: str, source: str):
    with open(chemin_fichier, "r") as f:
        texte = f.read()

    from langchain_text_splitters import RecursiveCharacterTextSplitter
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
    chunks = splitter.split_text(texte)

    embeddings = [generer_embedding(chunk) for chunk in chunks]

    collection.add(
        documents=chunks,
        embeddings=embeddings,
        ids=[f"{source}_{i}" for i in range(len(chunks))]
    )
    print(f"{len(chunks)} morceaux indexés depuis {source}")

```

6.3 — Génération de réponse avec le modèle local

python

```
def demander_a_assistant_local(question: str) -> str:
    embedding_question = generer_embedding(question)
    resultats = collection.query(query_embeddings=[embedding_question], n_resul
chunks_pertinents = "\n---\n".join(resultats["documents"][0])

    prompt = f"""
    Extraits de documentation :
    {chunks_pertinents}

    Question : {question}

    Réponds en te basant uniquement sur ces extraits.
    """
    reponse = ollama.chat(
        model="qwen2.5-coder:7b",
        messages=[{"role": "user", "content": prompt}]
    )
    return reponse["message"]["content"]
```



Avantage énorme pour ton stage : ce pipeline RAG tourne **entièrement en local**, sans clé API, sans coût, sans limite de requêtes. Tu peux tester autant que tu veux pendant ton apprentissage, et ne basculer vers l'API Claude que pour la version finale si tu veux une meilleure qualité de réponse.



Terminé quand :

```
python
indexer_document("docs/erreur_test.md", "test")
print(demander_a_assistant_local("Explique l'erreur"))
```

retourne une réponse cohérente, sans connexion Internet.

ÉTAPE 7 — Organiser ton dataset (la documentation pour le RAG)

7.1 — Créer la structure de dossiers

```
bash
mkdir docs
cd docs
mkdir erreurs_docker erreurs_mqtt erreurs_zephyr zephyr_officiel
```

7.2 — Où trouver du vrai contenu à indexer

Source	Où la trouver	Comment la récupérer
Doc officielle Zephyr	https://docs.zephyrproject.org	Copie les pages "Getting Started" et "Application Development" en <code>.md</code> ou <code>.txt</code>
Erreurs Docker rencontrées	Vos propres erreurs (toi, Ameni, Nour)	Copie le message d'erreur exact + la solution trouvée, dans un fichier <code>.md</code>
Erreurs MQTT/Mosquitto	Vos propres erreurs	Idem
Documentation <code>west</code>	https://docs.zephyrproject.org/latest/develop/west/index.html	Copie les commandes principales et leur usage

7.3 — Format recommandé pour chaque fichier

markdown

Titre court de l'erreur ou du sujet

Contexte

Dans quelle situation ça arrive.

Solution / Explication

La solution exacte, étape par étape.

Ce format aide beaucoup le chunking (Étape 6) à découper proprement, parce que chaque section a un sens complet.

✅ **Terminé quand :** tu as au moins 5-6 fichiers `.md` dans `docs/`, remplis avec du vrai contenu (pas des placeholders vides).

ÉTAPE 8 — Commit et push ton travail sur GitHub

```
bash

git add .
git commit -m "Setup initial : Docker, Ollama, RAG local, structure docs"
git push origin main
```

✅ **Terminé quand :** tu vois tes fichiers apparaître sur la page GitHub de ton dépôt dans le navigateur.

Récapitulatif : ton setup complet et 100% gratuit

```
VS Code (éditeur)
├── Extension Cline → Ollama (localhost:11434)
│   ├── qwen2.5-coder:7b (code + chat)
│   └── nomic-embed-text (embeddings RAG)
├── Extension GitHub Copilot Free (complément autocomplétion)
├── Extension Docker (gestion visuelle des conteneurs)
└── Git → GitHub (ton dépôt distant)

Docker Desktop
└── docker-compose.yml → backend, Mosquitto, InfluxDB, Grafana

ChromaDB (local, dossier ./rag_db)
└── alimenté par les embeddings d'Ollama (gratuit, illimité)
```

Coût total : 0€. Aucune carte bancaire nécessaire à aucune étape.

Pour toute question sur une étape précise, dis-moi laquelle exactement et je détaille encore plus.